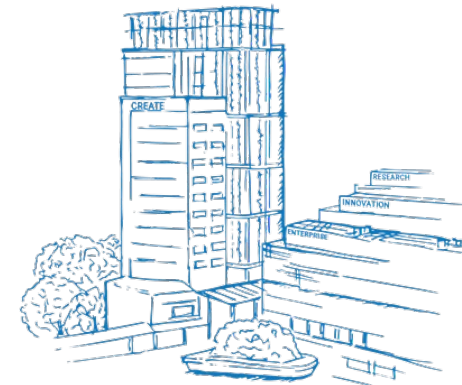


Agentic AI Workshop

Week 3 · From Loops to Graphs: stateful agents, multi-agent systems, deep research and agentic harness engineering.

Dr. Kerem Delikoyun · TUMCREATE · 10 June 2026



BLOCK 0 · 10 MIN

The one way loop

Last week: The control loop WAS the agent.

Today: We will look beyond what a bare `while` loop can't do but the data structure that can.

Week 2 recap, in one slide

What you built last week

- **Tool use**, if `model.has_tool_call`: `run(it)` in a loop
- **ReAct**, Thought → Action → Observation, by hand then `create_react_agent`
- **RAG**, chunk → embed → retrieve → augment the prompt
- All on **classic LangChain**, the last line with `AgentExecutor`

Week 2's ReAct, hand-rolled (NB02)

```
# the control loop IS the agent
scratchpad = ""
for _ in range(MAX_STEPS):
    out = llm(prompt(q, scratchpad),
               stop=["\nObservation:"])
    act = parse_action(out) # Action+Input
    if act is None:
        return out # Final Answer
    obs = TOOLS[act.name](act.arg)
    scratchpad += out + f"\nObs: {obs}\n"
```

We keep that honesty this week. The next abstraction has to earn its place, so we start by pushing a plain loop until it breaks.

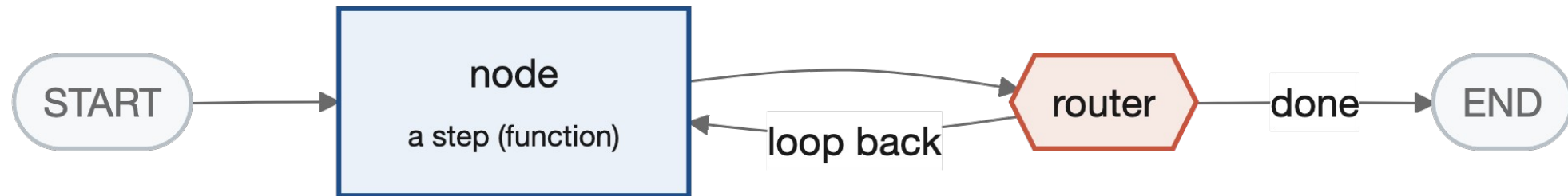
Four things a `while` loop can't do cleanly

- **Cycles with state**, reflect → revise → reflect, each turn reading *structured* state, not a string scratchpad
- **Conditional routing**, "is this retrieval relevant? if not, web-search instead", branching a linear chain can't express
- **Pause / resume / persist**, a loop dies when the cell ends; an agent that asks a human to approve must freeze and wait
- **Parallel fan-out**, run three retrievals at once, fan the results back in

All four are clumsy or impossible to hand-roll in a plain loop, yet each falls out naturally once you model the agent as a **graph**.

```
# each limitation maps to one graph primitive
cycle   : g.add_edge("revise", "reflect")      # a back-edge
branch  : g.add_conditional_edges("grade", route) # if / elif
pause   : compile(interrupt_before=["tools"])   # freeze, resume
parallel: return [Send("worker", x) for x in items] # fan-out
```

Here is LangGraph: the agent as a stateful graph



Nodes = steps (plain functions). Edges = control flow (add_edge is a line; add_conditional_edges is if; an edge pointing back is while).

Why a graph, not a chain?

LangChain (LCEL) chains are DAGs

- A chain is a **directed acyclic graph (DAG)**: input → step → step → output, no edge points back
- Perfect for a fixed flow (a RAG query, a summariser) that never loops
- No native cycle, no pause, no shared mutable state across steps
- AgentExecutor bolted a loop on top: opaque, hard to inspect, hard to resume

a chain = a DAG (no edge ever points back)



LangGraph doesn't replace LangChain. Chains still build your nodes. The graph is the control layer around them.

THE GRAPH TEST

If your agent ever needs to **loop**, **branch on a condition**, **pause for a human**, or **remember across turns**, you've outgrown a chain. That is exactly what a graph gives you, explicitly.

The agent-framework landscape

Framework	Control model	State / memory	Best when
LangGraph	explicit graph (nodes + edges), low-level	typed state + checkpointer + store	you need control, cycles, persistence, prod reliability
CrewAI	role-based crews + tasks (high-level)	task context passing	a fast role/task team demo, less custom control
AutoGen / AG2	conversational agents (message passing)	shared chat history	conversational + code-exec research agents
OpenAI Agents SDK	agents + handoffs + guardrails (light)	sessions + built-in tracing	OpenAI-centric, simple handoff orchestration
LlamaIndex Workflows	event-driven steps	index / context-centric	RAG-heavy, data-framework-first apps

Rule of thumb: CrewAI / AutoGen to stand up a multi-agent demo fast; LangGraph when you must control the loop, persist state, and ship it. They are not exclusive, you can wrap one inside the other.

The version cutover, and why it matters

Week 2 → Week 3

- **Week 2** pinned **classic LangChain**, the last line with AgentExecutor
- **Week 3** moves to **LangGraph**, AgentExecutor is legacy; the graph is now the supported path
- StateGraph + ToolNode replace the create_react_agent black box, every node and edge is explicit
- Concepts carry over one-to-one; only the imports change

Same free **Gemini** provider (gemini-2.5-flash), same Colab-first delivery.

A RELIEF FROM LAST WEEK

LangSmith tracing *just works* every call routes through a Runnable, so the tracer attaches automatically.

PART 1 · NOTEBOOK 00

LangGraph core architecture and concepts

State, nodes, edges, the node taxonomy, and how memory flows: the spine of every agent this week.

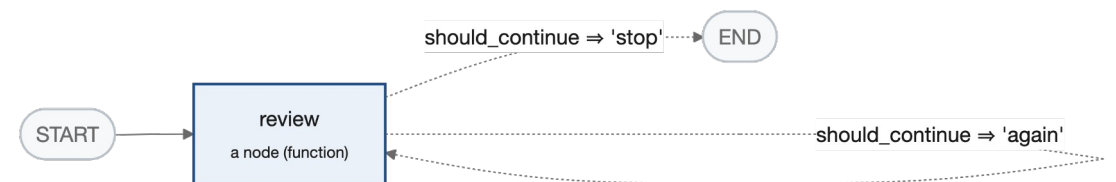
A node is a function. An edge is control flow.

The whole API, once

- **State**, one typed dict every node reads/writes;
Annotated[list, add_messages] makes a field *accumulate*
- **Node**: `def f(state) -> partial_state`. No magic
- **Edges**: `add_edge` (line), `add_conditional_edges` (router), `back-edge` (loop)
- `compile()` → `.invoke()` / `.stream()`

```
g = StateGraph(State)
g.add_node("review", review)
g.add_edge(START, "review")
g.add_conditional_edges(
    "review", should_continue,
    {"again": "review", "stop": END},
)
app = g.compile()
```

what add_conditional_edges builds



NB00 builds this with NO LLM first (pure-Python nodes), so the graph is transparent before any model is involved.

State: the typed dict every node shares

```
class State(TypedDict):  
    messages: Annotated[  
        list, add_messages]  
    draft: str  
    score: int  
    iterations: int
```

Three ideas do all the work

- **State is one TypedDict**, every node reads it and returns a few keys to update
- **Channels**: each key is a slot; messages uses `add_messages` so turns *append*, not overwrite
- **Reducer**: only needed when several nodes write the same key at once; it says how to merge (else last write wins)
- **You rarely need more**, most agents are messages plus a couple of scalars

Pick your reducer deliberately: it is the single most common source of "my parallel nodes clobbered each other" bugs.

Node & edge types you'll actually use

Primitive

[node \(function\)](#)

[ToolNode](#)

[add_edge](#)

[add_conditional_edges](#)

[Send](#)

[START / END](#)

[subgraph](#)

What it is

`def f(state) -> partial`

prebuilt node that runs tool calls

unconditional line $A \rightarrow B$

router fn $\rightarrow$ next node name

dispatch dynamic parallel work

entry + terminal sentinels

a compiled graph used as a node

Use it for

any step: call an LLM, run code, transform state

execute whatever the LLM requested

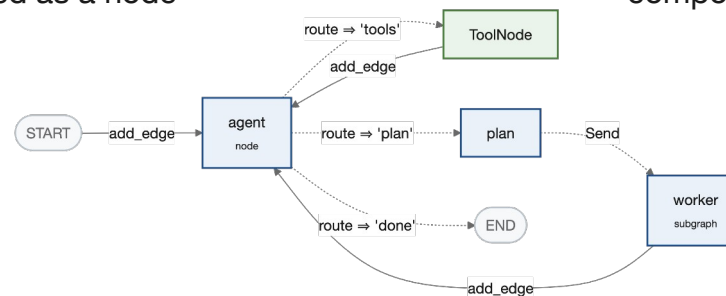
a fixed sequence

branch / loop: the `if` and the `while`

map-reduce fan-out (one item $\rightarrow$ one worker)

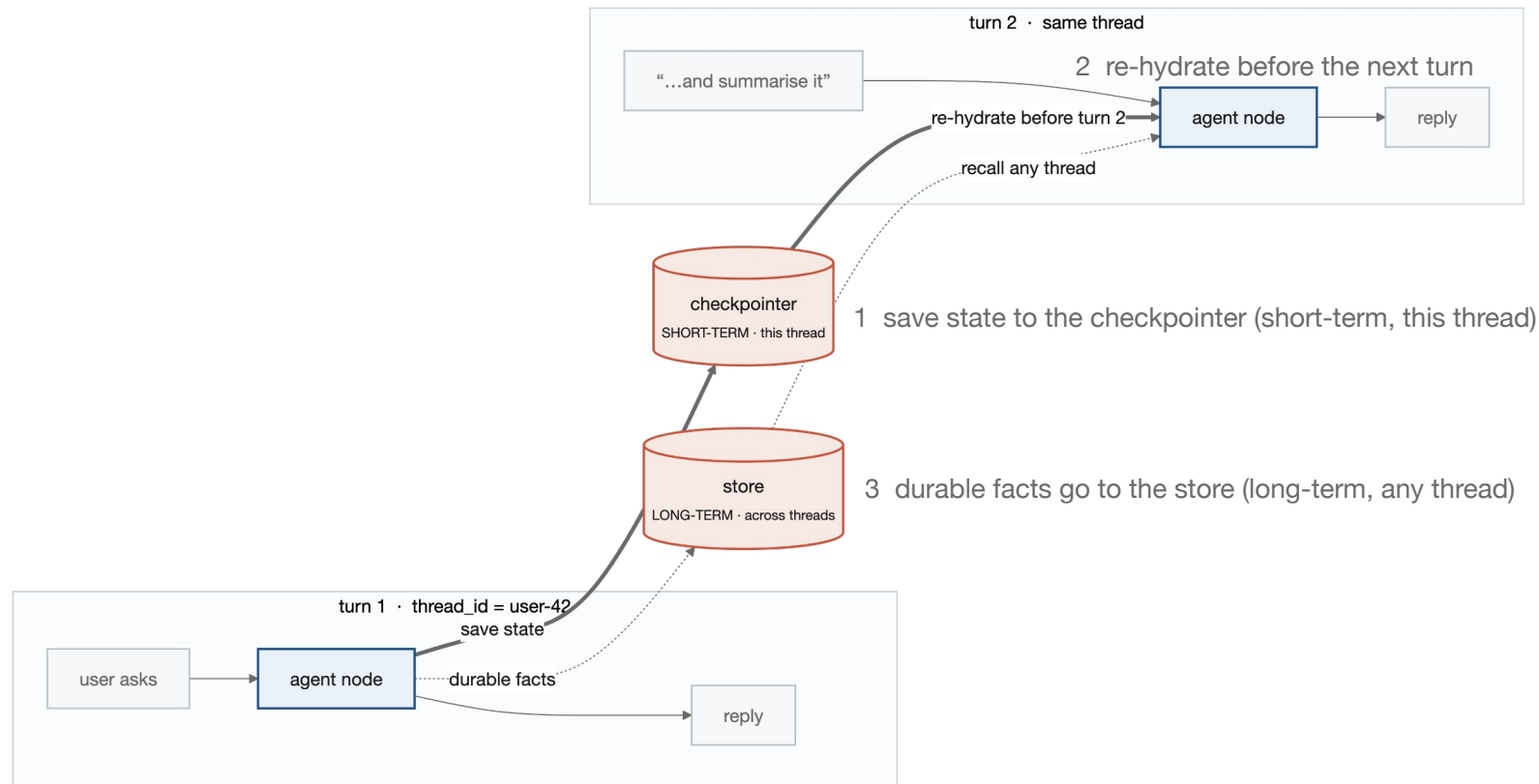
wire the graph boundaries

compose / multi-agent



Seven primitives. Every pattern this week (reflexion, RAG, supervisor/worker) is a recombination of these.

Memory & conversation flow



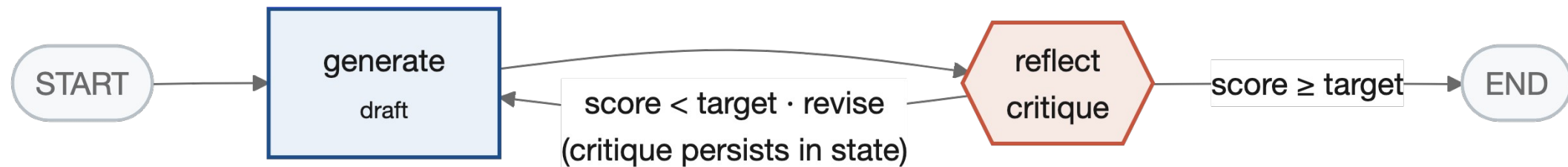
Short-term = thread state via the checkpointer (survives turns and kernel restarts). Long-term = the store (facts that cross threads). Messages accumulate via `add_messages`; everything else you design.

PART 2 · NOTEBOOK 01

Reflexion

Generate → reflect → revise, a self-improving loop that measurably sharpens a paper abstract.

Reflexion: critique that persists



Reflection = generator + critic. Reflexion (Shinn et al., 2023) keeps the critique in state so each revision targets prior weaknesses.

Reflection vs Reflexion: the mechanism

Reflexion ([Shinn et al., 2023](#))

- Actor generates an attempt
- Evaluator scores it: here, a structured critique
- Self-reflection turns that score into verbal feedback
- Written to state as **episodic memory**: a running log of *this run*'s past attempts and their critiques (not general facts); the next attempt reads it back

VERBAL REINFORCEMENT

No gradient updates, no fine-tuning. The model improves *within a session* because its own critique becomes part of the next prompt. Cheap, fast, fully traceable.

Plain reflection = generator + critic, one shot. Reflexion = reflection that *persists and accumulates*, so the output converges instead of drifting.

The critic returns a number you can branch on

The reflect node uses **structured output**. Gemini fills a Pydantic schema, so you get a score, not prose. Keep the model **flat** (nested/Union can fail Gemini's JSON mode).

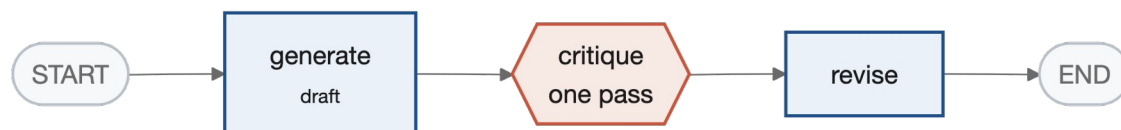
- Loop ends when **score $\geq$ target** or **max iterations**
- Maps to research writing: draft → reviewer critique → revise
- Watch the score climb turn by turn in LangSmith

```
class Critique(BaseModel):  
    score: int  
    suggestions: list[str]  
  
critic = llm.with_structured_output(Critique)
```

Reflection vs Reflexion: code and graph, side by side

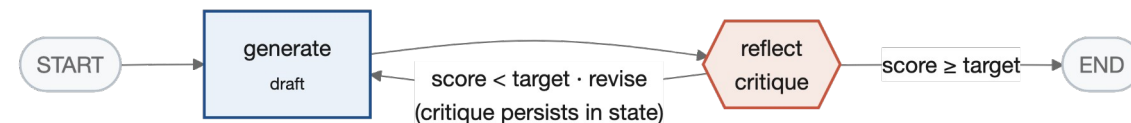
Reflection: one pass · Self-Refine ([Madaan et al., 2023](#))

```
# the critique is discarded after one pass
g.add_edge("generate", "critique")
g.add_edge("critique", "revise")
g.add_edge("revise", END)
```



Reflexion: persists + loops · ([Shinn et al., 2023](#))

```
# the critique persists in state, then loops
g.add_conditional_edges("reflect",
    should_continue,
    {"again": "generate", "stop": END})
```



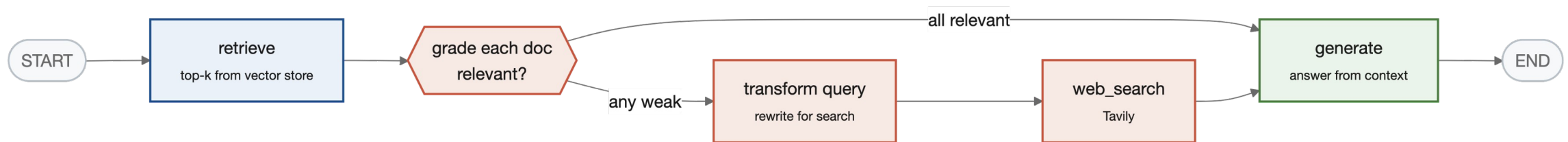
Plain reflection runs once; Reflexion keeps the critique in state and loops, so the output converges instead of drifting.

PART 3 · NOTEBOOK 02

Agentic RAG

Retrieval that grades itself and web-searches the gaps: RAG that knows when it doesn't know.

Corrective RAG: grade, then maybe go fetch



The grader is the agentic part: a structured yes/no per document decides whether to trust the corpus, or rewrite the query and fall back to web search.

Why agentic RAG over naive RAG?

Where naive RAG breaks

- Retrieves k chunks and **trusts them**, even when off-topic
- No recourse when the corpus simply lacks the answer
- Hallucinates confidently over weak context
- One static path: retrieve → stuff → generate

What the grader adds

- A structured **relevant? yes/no** per document
- Weak → **rewrite the query and web-search** instead of guessing
- (Self-RAG) also grade the *answer*: grounded? does it answer?
- Retrieval becomes a **decision**, not a reflex

Same retriever, same model. The difference is a conditional edge that lets the graph notice it doesn't know.

Three patterns, one graph

Pattern	What it adds	Extra nodes
Corrective RAG	grade retrieved docs; web-search fallback	grade_documents → web_search
Self-RAG	also grade the <i>generation</i> : grounded? answers?	grade_generation (hallucination + usefulness)
Adaptive RAG	route the query first: direct / vectorstore / web	a router node off START

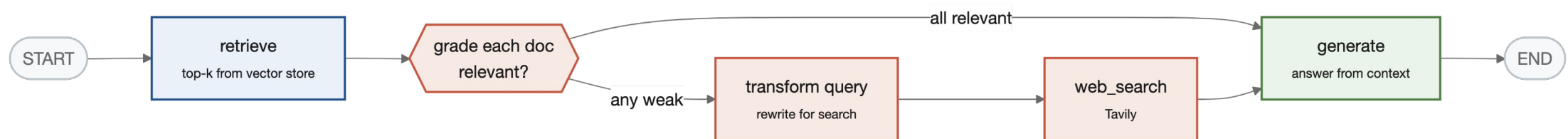
Same primitives (a conditional edge and a grader) composed differently. NB02 builds CRAG fully and bolts on the rest.

Naive vs agentic RAG: the architectures

Naive RAG: retrieve → stuff → generate. One static path; it trusts whatever comes back.



Agentic RAG (Corrective): grade each doc, then rewrite + web-search the gaps.



```
# the whole difference is one conditional edge:  
g.add_conditional_edges("grade", route,  
    {"relevant": "generate", "weak": "web_search"})
```

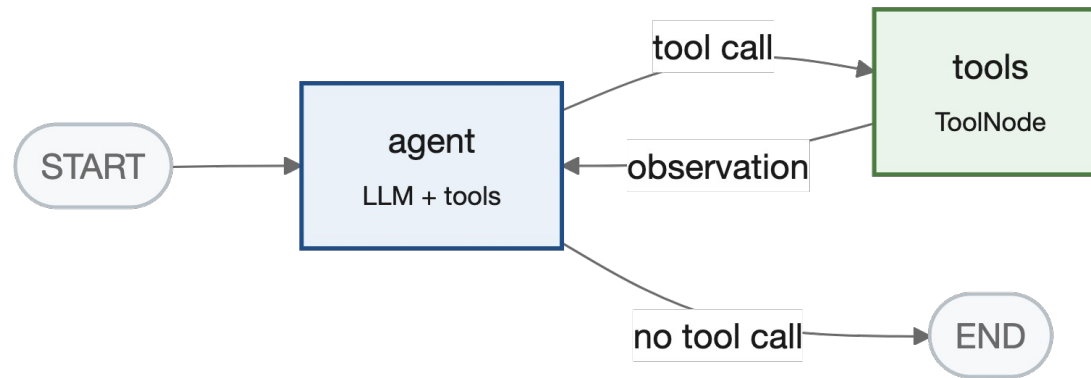
Same retriever, same model. One conditional edge lets the graph notice when it doesn't know.

PART 4 · NOTEBOOK 03

From demo to real world

Memory across turns, a pause for human approval, durable state, and parallel fan-out: each one small additive step.

Week 2's ReAct loop, now two nodes



```
# LangChain
agent = create_react_agent(
    llm, tools)
AgentExecutor(agent=agent,
    tools=tools).invoke(...)

# LangGraph: explicit
g.add_node("agent", call_llm)
g.add_node("tools",
    ToolNode(tools))
g.add_conditional_edges(
    "agent", tools_condition)
g.add_edge("tools", "agent")
```

`ToolNode` runs whatever tools the agent called; `tools_condition` closes or continues the loop. Same control flow, now inspectable and pausable.

Persistence + human-in-the-loop

Add a checkpointer

- `MemorySaver + thread_id` → memory across turns (a follow-up with a pronoun resolves)
- `SqliteSaver` → durable: the run survives a kernel restart, timeout, or closed laptop. Resume from the last checkpoint
- `interrupt_before=["tools"]` → the graph pauses before acting; a human approves, you resume with `invoke(None, cfg)`
- **Why a human gate:** for irreversible or costly actions (DB writes, emails, spend, long compute), approval before acting is how you keep control and earn trust

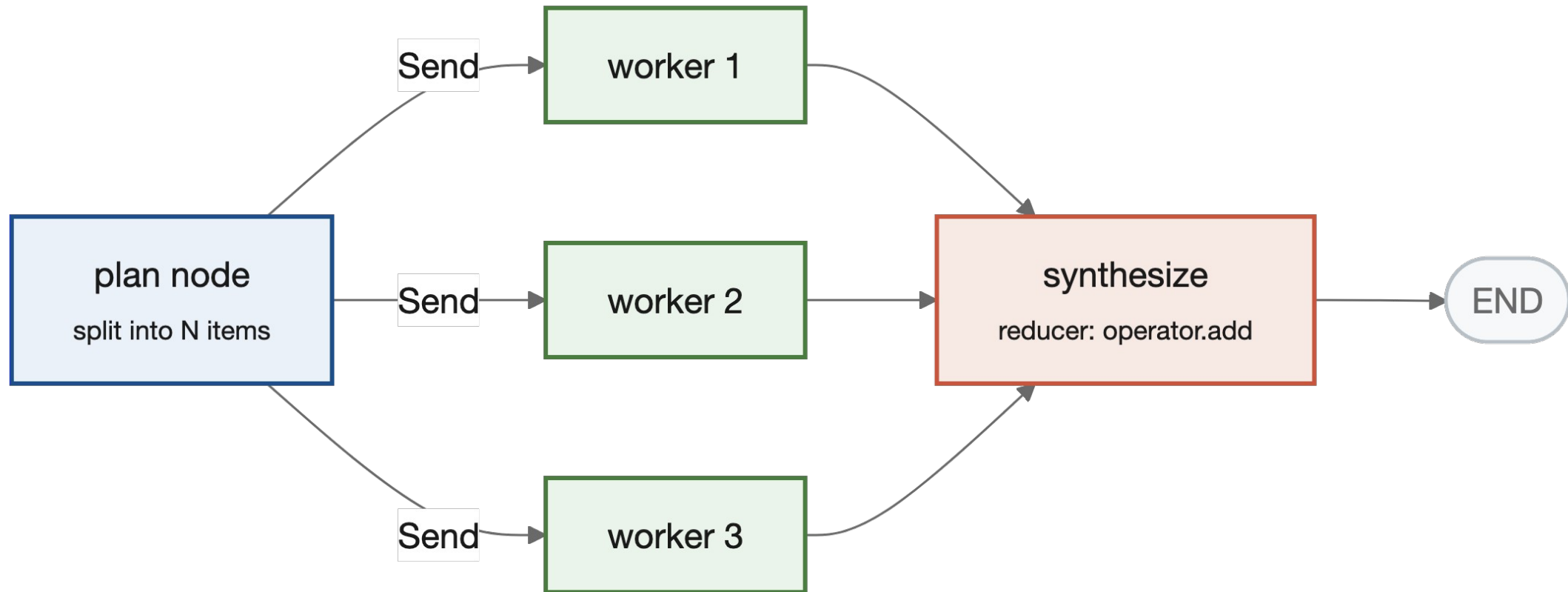
WHY SCIENTISTS CARE

An assistant that *remembers* your last question and *asks permission* before it writes to a database, emails a collaborator, or kicks off a long compute job.

```
app = build().compile(  
    checkpointer=MemorySaver(),  
    interrupt_before=["tools"],  
)
```

Pause-inspect-resume is impossible with a plain loop. It needs the checkpointer to freeze state mid-graph.

Async parallel fan-out / fan-in



Send dispatches one worker per item; a **reducer** (`operator.add`) merges their writes instead of clobbering. Wall-clock drops to the slowest branch, not the sum, but every node on the path must be async-safe.

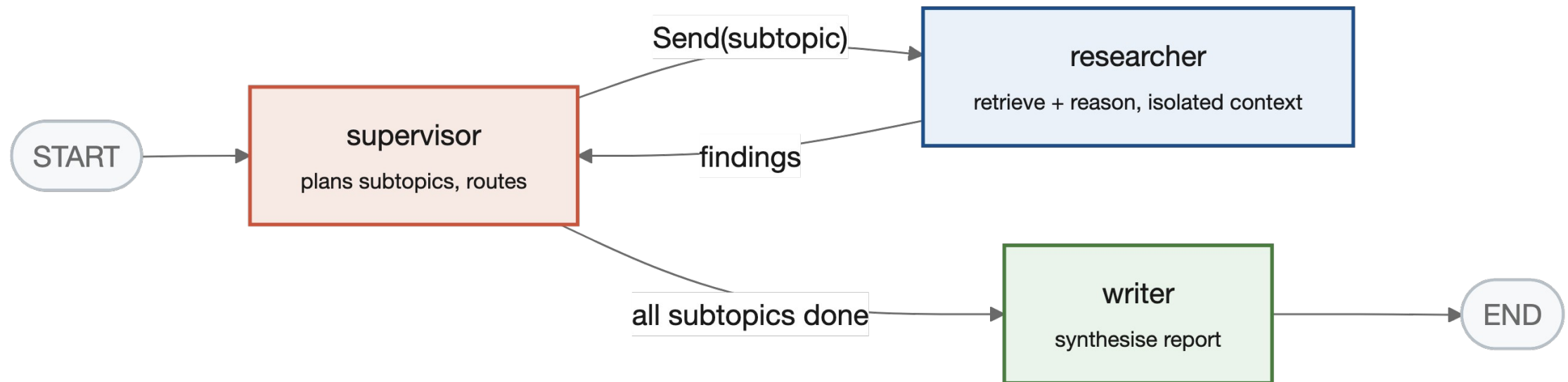
PART 5 · NOTEBOOK 04

Multi-agent systems.

Case study: Deep research

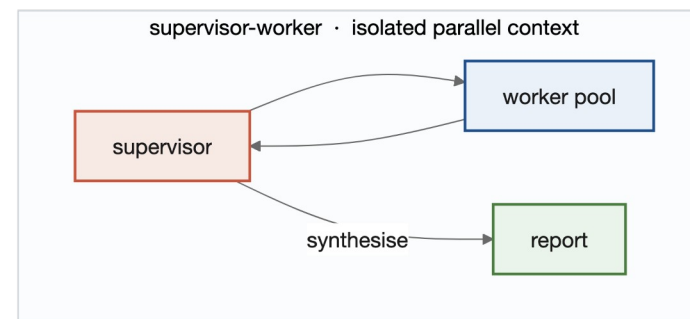
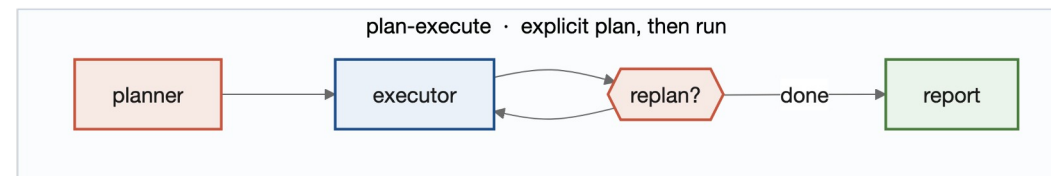
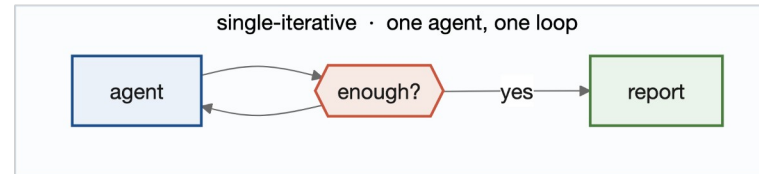
When one agent's context/expertise isn't enough: a supervisor delegating to sub-agents.

Supervisor → sub-agents



Multi-agent is *specialisation, not headcount*. Each worker gets its own isolated context window; the supervisor only sees their findings, not their scratch work.

Three deep-research architectures

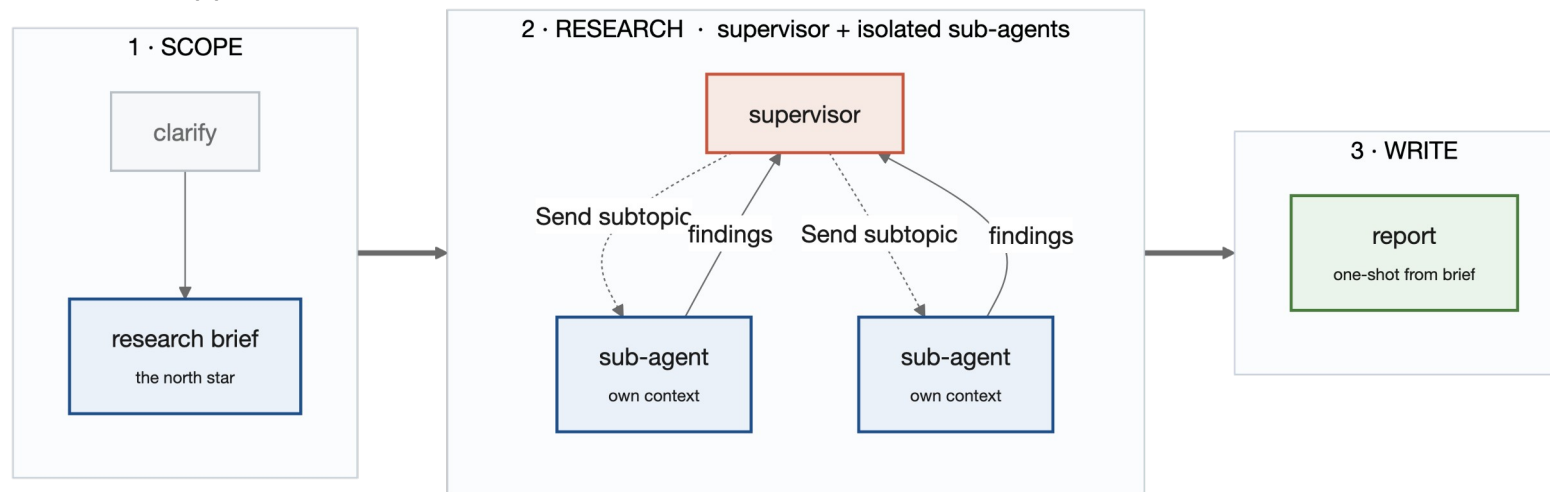


LangChain's **Open Deep Research** is this Scope → Research → Write shape (next slide): a supervisor delegates to **context-isolated** sub-agents in parallel, then writes the report **one-shot** from the brief. They found parallel *writing* produced disjoint sections.

The real systems use this exact shape

System	Shape	What it adds
open_deep_research	single-iterative · plan-execute · supervisor-worker	per-role models via <code>init_chat_model()</code> ; HIL approval
GPT-Researcher	planner → parallel executors → publisher	STORM-style report generation
TriAgent (ours)	4-node pipeline; supervisor/worker deep research	clinical biomarker discovery + literature validation

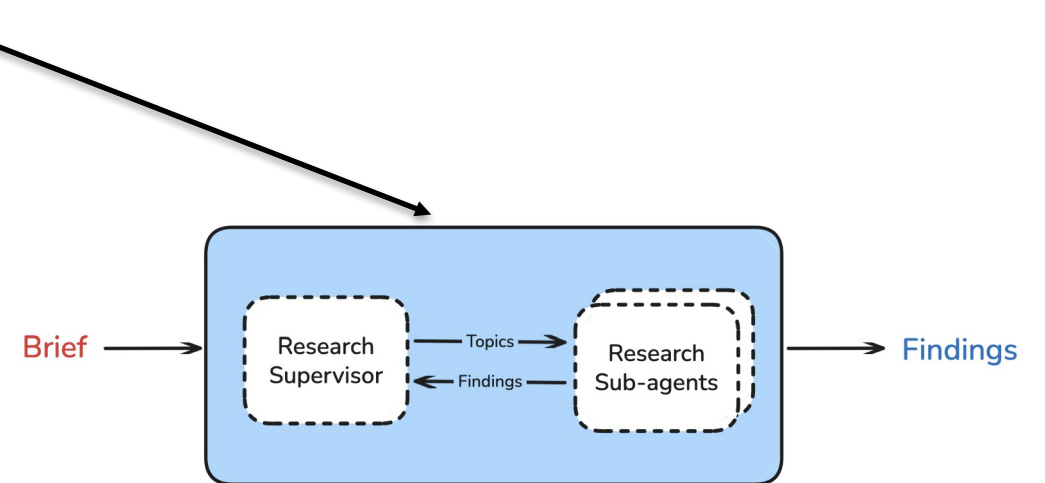
All recombinations of the Part-1 primitives. Open Deep Research's lesson: use multi-agent only for **easily-parallelised** work (research), keep writing single-pass, and expect ~15× the tokens of a chat app.



Open Deep Research (LangChain): Scope → Research → Write



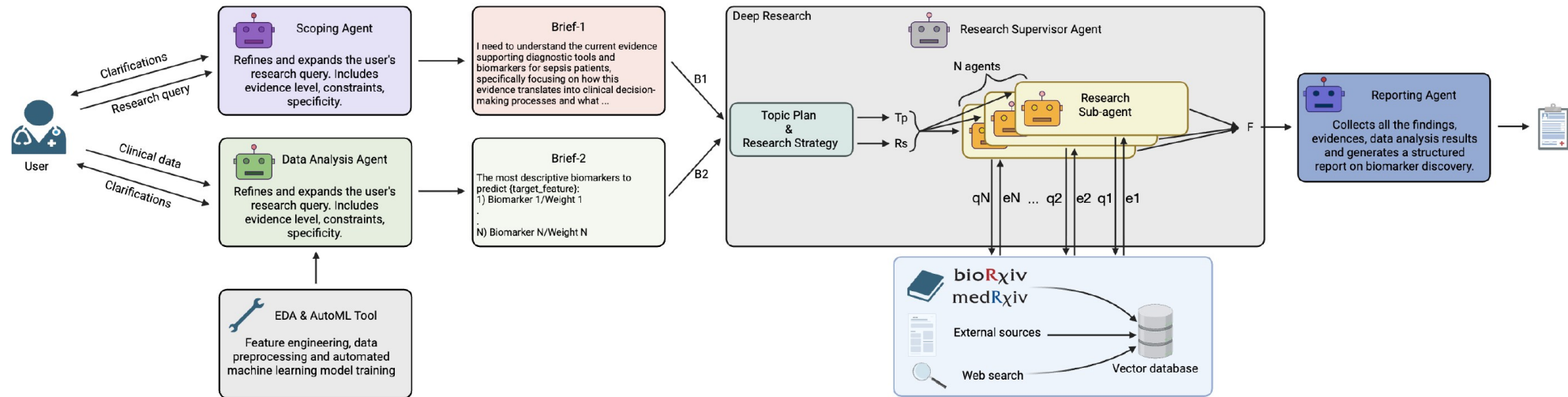
- **Scope:** clarify with the user, then write a research **brief** that stays the success metric
- **Research:** a **supervisor** delegates sub-topics to **context-isolated** sub-agents, in parallel
- **Write:** one LLM call turns the brief + all findings into the report, in a **single pass**



Their lesson: keep multi-agent for research only and write the report one-shot. Parallel section-writing produced disjoint reports. Context isolation curbs token bloat; deep research can use ~15× the tokens of a chat app.

Source: LangChain, “Open Deep Research”, langchain.com/blog/open-deep-research

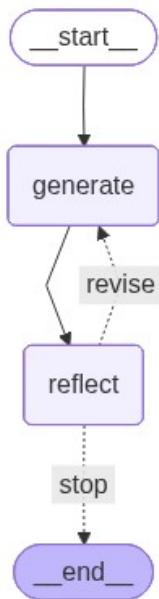
TriAgent: biomarker discovery through automated data analysis and deep research



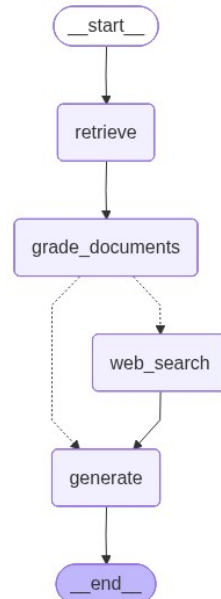
Delikoyun et al., "TriAgent" (arXiv:2510.16080).

What your notebooks bring

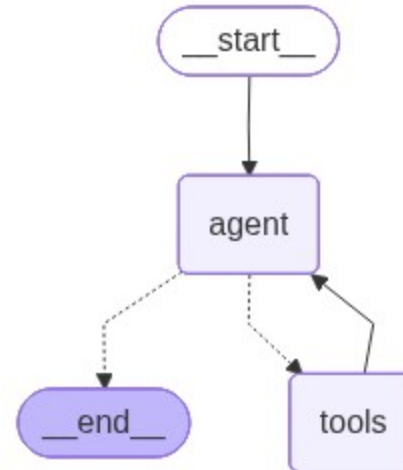
Every agent you build compiles to a graph, and the app renders its own topology. This is the live output, not a redrawing.



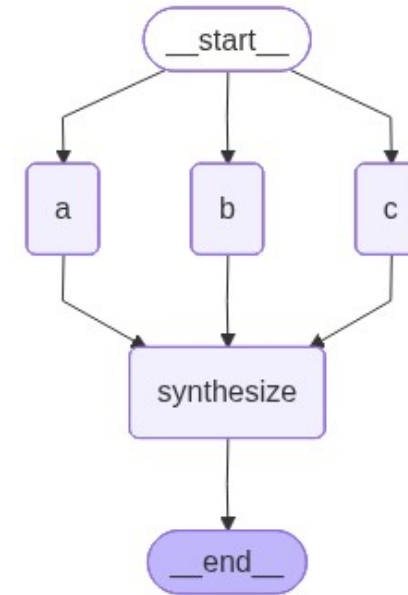
01 · Reflexion



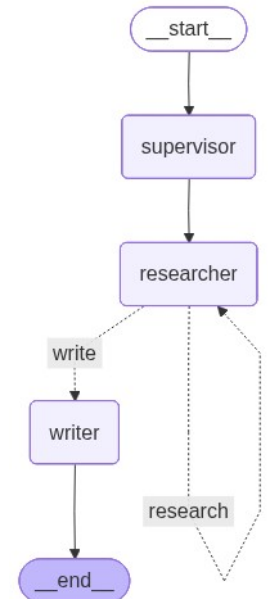
02 · Agentic RAG



03 · ReAct loop



03 · Fan-out



04 · Deep research

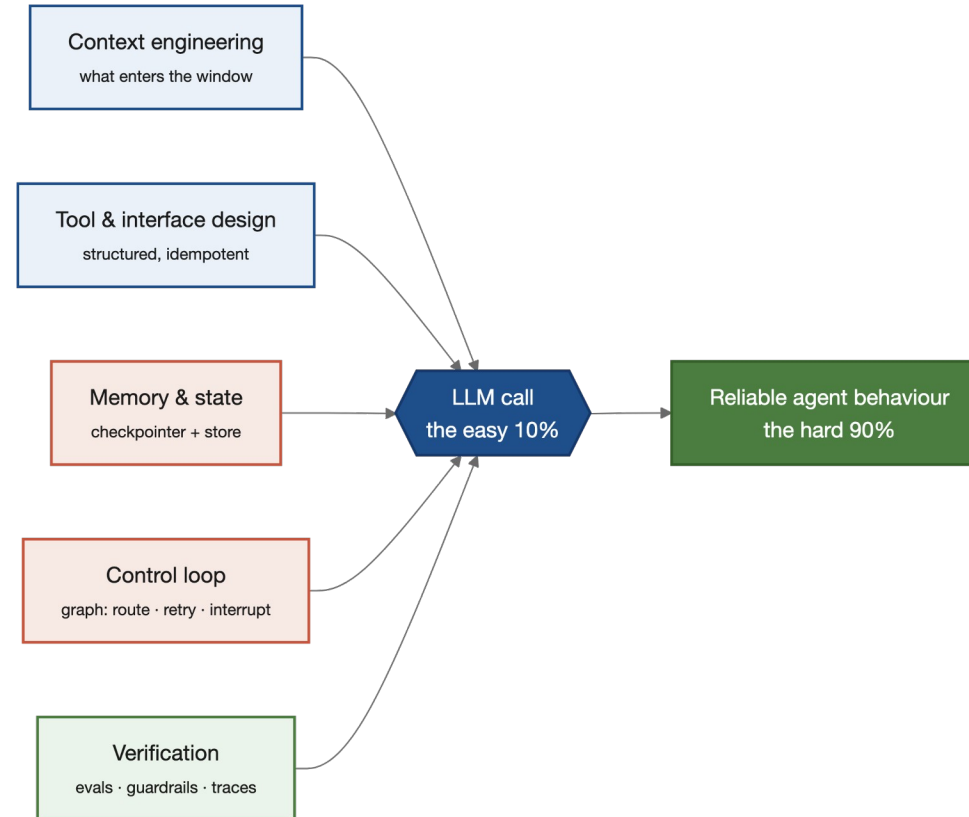
These are the control flows you wired: reflect loops, grade-then-fetch branches, agent/tool cycles, parallel fan-out, and supervisor routing.

PART 6 · 10 MIN

Agentic Harness Engineering

Picking a smarter model rarely fixes a flaky agent. What makes it reliable is the code around the call: context, tools, memory, control flow, verification. Teams have climbed from top-30 to top-5 on a benchmark by changing only that layer. It is called the harness, and engineering it is most of the work.

What is a harness?



Swapping in a smarter model rarely fixes an unreliable agent. The reliability lives in everything around the call. The harness is an ecosystem the model plugs into: change the LLM and its raw ability changes, but the harness is what lets any model actually reach those capabilities.

Sources: "Agent Harness Engineering" Lin et al., arXiv:[2604.25850](https://arxiv.org/abs/2604.25850)

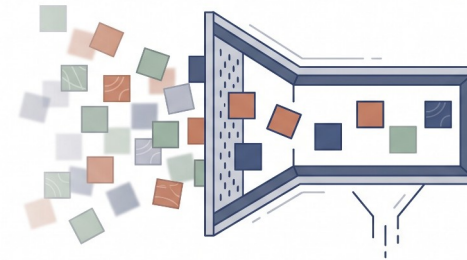
Context engineering

The window is the bottleneck

- Every token competes: **relevance beats volume**
- **Retrieve just-in-time**, don't pre-stuff the prompt
- **Compact**: summarise old turns, drop dead context
- **Isolate**: give each sub-agent only its slice (the supervisor/worker win)

THE FAILURE YOU'LL SEE

An agent that was sharp at turn 2 goes vague at turn 20. It's not the model: it's a window full of stale context. Engineering what stays in is half the job.

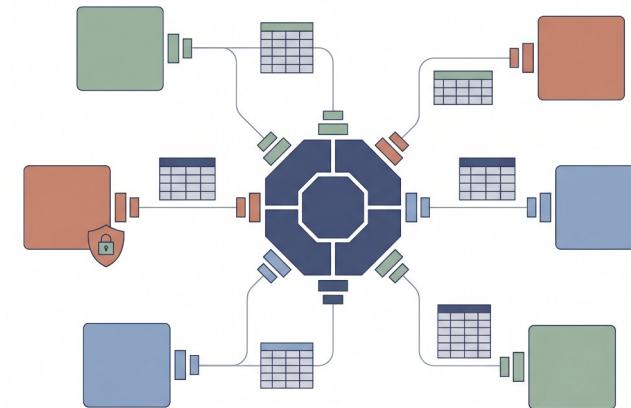


Tool & interface design

Tools are the agent's API to the world

- Return **structured** results the model can branch on, not prose
- Make them **idempotent**: agents retry
- Fail **loudly and usefully**: an error string the model can recover from
- **Least privilege**: a tool that can delete needs a human gate

```
@tool
def search(q: str) -> list[dict]:
    """Up to 5 {title,url,snippet}."""
    ...
# typed in, structured out,
# safe to retry
```



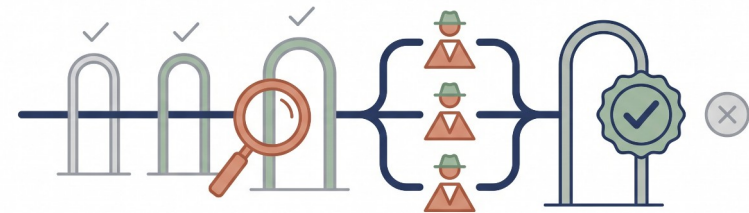
Verification & evaluation loops

Trust, but verify, automatically

- **Eval gates:** a held-out set the agent must pass before ship
- **LLM-as-judge** for open-ended output; verify the risky ones multiple ways
- **Guardrails:** schema-validate every structured output
- **Trace everything** (LangSmith): you can't fix what you can't see

A HABIT THAT TRAVELS

The discipline that wins hard problems: verify a result several independent ways before you trust it. The same habit turns a flaky agent into a dependable one.



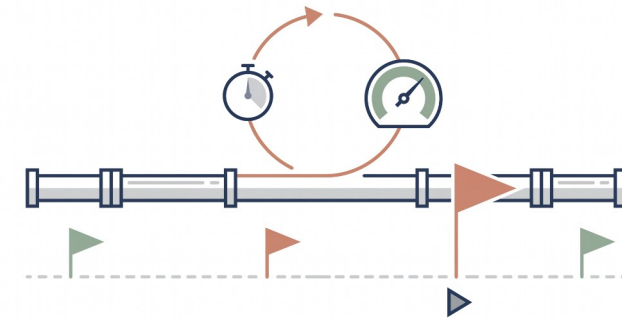
Reliability & observability

What makes it production, not a demo

- **Retries + timeouts** on every external call
- **Determinism where it matters:** pin temperature, seed, versions
- **Budgets:** cap tokens, steps, and cost per run
- **Checkpoints:** resume from the last good state, don't restart

THE MINDSET SHIFT

A demo only has to work once, while you're watching. Production has to work on the hundredth run, on the input you didn't expect, when nobody is awake to restart it. Closing that gap is engineering, not prompting.



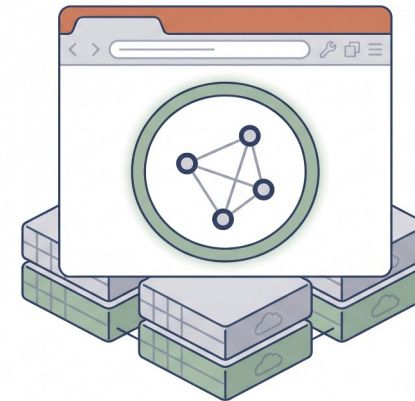
Deployment: a tour (not hands-on today)

Wrapping a compiled graph

- **LangGraph Studio**, a visual IDE: run, inspect, and time-travel through your graph's state
- **LangGraph Cloud / Platform**: the graph as a hosted API (assistants, threads, runs)
- **LangSmith**: tracing + evaluation in production
- **Self-host / cloud**: same compiled graph behind FastAPI on **Ray Serve**, **Ollama**, managed on **AWS Bedrock AgentCore**; **Unsloth** for fine-tuned local models

THE POINT

Everything today compiles to one graph object. Deploying it just wraps that object in an API and a UI. The graph you tested is the graph that ships.



CLOSE · 5 MIN

Run it yourself

Five notebooks back every idea today. The links land in chat: fork them, break them, make them yours.

The five notebooks

- [00 · LangGraph core](#): state, nodes, edges, the conditional-edge cycle
- [01 · Reflexion](#): generate → reflect → revise, until good enough
- [02 · Agentic RAG](#): retrieve → grade → web-search the gaps
- [03 · Production](#): memory, human-in-the-loop, durable state, parallelism
- [04 · Multi-agent](#): supervisor + workers → the shape behind real deep-research systems

► [Open all in Colab · github.com/krmde/agent-ai-for-scientists-workshop → week-03-multi-agent/notebooks](#)

Click any notebook title above to open it in Colab. Build the graph well, then engineer the harness around it: that's the whole job. Week 4 builds on these toward evaluation and domain agents.